

PROGRAMMING IN C – UNIT 4

2 MARKS

1) List any two advantages of using functions?

Code Reusability: Function can be called or used as many times as required which reduces code repeatability and increases code reusability.

Reduction in Program Size: Functions can be called as many times as required. This avoids writing of same code again and again reducing program size.

Reducing Complexity of Program: Complex programs can be divided into small sub-programs or user defined functions.

2) Give the syntax used for declaring a function in C.

Syntax for declaring a function :

return_type function_name(parameter list);

return type is the data type of the output that is being returned. Function name is the name of the function. Parameter list depends on the number of input values that is being passed.

3) Give the syntax used for defining a function in C.

return_type function_name(parameter list)

```
{  
local variable declaration;  
statement1;  
statement2;  
.  
.  
return statement;  
}
```

The first line return_type function_name(parameter list) is the function header and the statements within the opening and closing braces is the function body.

4) What are the actual and formal arguments?

When a function is called, the values that are passed in the function call are called the arguments or actual parameters. The parameter used in function definition statement which contain data type on its time of declaration is called formal parameters.

5) How function returns value ? Give example.

Function returns value to the main function using the the keyword return. A return statement returns the value evaluated by the function.

Example :

```
int addition(int a , int b)  
{  
int res;  
res=a+b;  
return res;  
}
```

Here the return statement returns the result of sum of 2 values.

6) Differentiate function declaration and function definition.

Function declaration indicates what the function is and Function Definition indicates what the function does.

Function declaration is a prototype that specifies the function name, return types and parameters without the function body. Function Definition, on the other hand, refers to the actual function that specifies the function name, return types and parameters with the function body.

The function declaration helps to indicate about the function and how to call that function, etc. while Function definition helps to write what the function should perform. It is the actual implementation of the function.

7) List different categories of functions available in C.

Category 1 : Functions with no arguments & no return values

Category 2 : Functions with arguments & no return values

Category 3 : Functions with no arguments but returns a value

Category 4 : Functions with arguments & return values

8) Give syntax and example of calling a function C.

Syntax : `function_name(argument_list);`

Example : `addition(a,b);`

We must pass the same number of arguments as it is declared in the function declaration.

9) What do you mean by nesting of functions? Give example.

A function that is called inside another function is called as the nested function.

Example:

```
void India( );
void America( );
main( )
{
    printf("Hello world\n");
    india( );
    return 0;
}
void India( )
{
    printf("I love my India\n");
    America( );
}
void America( )
{
    printf("America America");
}
```

10) What is a recursive function?

The function that calls itself is called as the recursive function.

Recursive functions can be used to solve problems where solutions are expressed by applying the same solutions repeatedly. When we write recursive function then we need to have an if statement to force the function to return. Otherwise the function will not return at all, it will repeatedly get executed.

11) Write recursive function to calculate factorial of a given number.

```
int factorial(int n)
{
    if(n==0 || n==1)
        return 1;
    else
        return(n*factorial(n-1));
}
```

12) What is structure? Why it is required?

Structure is a constructed data type that is used to group the elements of different data types.

Structure is required to organize complex data in a more meaningful way. It is used to represent a collection of data items of different types using a single name.

13) Give the general format used for defining a structure.

General form of structure definition is :

```
struct tag_name
{
    data_type    member1;
    data_type    member2;
    .....
    .....
};
```

14) Give the general format used for defining a union.

```
union tag_name
{
    data_type    member1;
    data_type    member2;
    .....
    .....
};
```

15) How to Access individual members of structure? Give example.

The members themselves are not variables. They should be linked to the structure variables in order to make them meaningful members. The link between a member and a variable is established using the member operator ‘.’ which is also known as dot operator. For example

book1.price

is the variable representing the price of book1 and can be used as any other ordinary variable

Example :

book1.title = “ Programming in C”;

this will assign the title “Programming in C” to book1.

16) What is union? How it is declared?

Union is a user-defined data type which is a collection of different variables of different data types in the same memory location.

Unions follow the same syntax as structures. Union is declared using the keyword union.

General form (Syntax):

```
union tag_name
{
    data_type    member1;
    data_type    member2;
    .....
    .....
};
```

Entire definition is considered as a statement, each member is declared independently for its name and type in a separate statement inside the template.

17) Differentiate structure and Union.

Keyword struct is used to declare the structure whereas keyword union is used to declare the union.

The main difference is in terms of storage. In structures each member has its own storage location, whereas all the members of union share the same storage location. That implies that although a union contains many members with different types, it can handle only one member at a time.

6 MARKS

1) Explain with example how to declare, define and call a function in C.

A function is a block of code that performs a particular task.

Three elements of user defined functions

- 1) Function declaration
- 2) Function call
- 3) Function definition

Function declaration :

Like variables even functions need to be declared before they are used in the program.

Function declaration consists of 4 parts :

- Function type (Return type)
- Function name
- Parameter list
- Terminating semicolon

General form :

return_type function_name(parameter list);

return type is the data type of the output that is being returned. Function name is the name of the function. Parameter list depends on the number of input values that is being passed.

Example :

int addition(int a,int b);

Here int is the datatype of the output that is the sum of two integers. addition is the name of the function. a and b are the integer type variables used to pass the input values to add two integers.

When the function does not take any parameters and does not return any value, then the form is:

void addition();

We can declare the user defined function above the main function or inside the main function.

Function call

A function can be called by just using the function name followed by a list of parameters or arguments inside the parentheses. Function can be called from anywhere in the program. The parameter list must not differ in function calling and function declaration.

Syntax : **function_name(argument_list);**

We must pass the same number of arguments as it is declared in the function declaration.

Example :

```
main()
{
int sum;
sum=addition(a,b);                      //function call
printf("Sum is %d", sum);
}
```

When the compiler encounters a function call, the control will be transferred to the function addition(). Then the function body will be executed one by one and the value is returned to the main function when the return statement is encountered. The value returned will be assigned to variable sum and is printed.

Function definition

Function definition includes the following elements:

- 1) Function name
- 2) Function type
- 3) List of parameters
- 4) Local variable declarations
- 5) Function statements
- 6) A return statement

All the six elements are grouped into 2 parts:

- Function header (first three elements)
- Function body (second three elements)

General form :

```
return_type function_name(parameter list)
{
local variable declaration;
statement1;
statement2;
.
.
return statement;
}
```

The first line **return_type function_name(parameter list)** is the function header and the statements within the opening and closing braces is the function body.

Function header consists of three parts : return type the data type of the output been returned, function name and the parameter list the input values.

The function body contains the declarations and statements necessary for performing the required task. The function body contains three parts :

- Local declarations that declares the variables needed by the function.
- Function statements that perform the task of the function.
- A return statement that returns the value evaluated by the function.

If the function does not return any value then we can skip the return statement. Its return type should be specified as void.

Example :

```
int addition(int a , int b)
{
int res;
res=a+b;           //function definition
return res;
}
```

**Example program to add two integers using a user defined function.
(Only if asked)**

```
int addition(int a , int b);           //function declaration
main()
{
int sum, a=2, b=4;
sum=addition(a,b);                   //function call

printf("Sum is %d", sum);
return 0;
}
int addition(int a , int b)           //function definition
{
int res;
res=a+b;
return res;
}
```

Here int addition(int a,int b) declares the function addition(). The function call sends the 2 integer values 2 and 4 stored in a and b variables respectively to the function. The function body adds the two integers and assigns it to res and then the value 6 is returned to the main function and assigns to the variable sum. Then the result will be printed.

2) Explain with example function with no arguments and no return values

3) Explain with example function with arguments and no return value

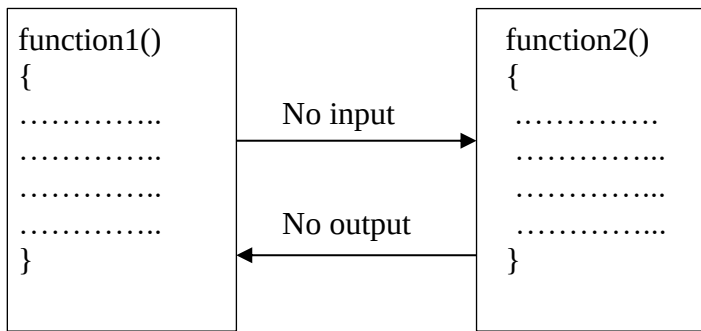
4) Explain with example function with no arguments but a return value

5) Explain with example functions with arguments and a return value

A function depending on whether arguments are present or not and whether a value is returned or not, may belong to one of the categories.

Category 1 : Functions with no arguments & no return values

When a function has no arguments it does not receive any data from the calling function. When it does not return a value the calling function does not receive any data from the called function. So there is no data transfer between the calling function and the called function.



Example :

```

void addition ();
main()
{
addition();
return 0;
}

void addition()
{

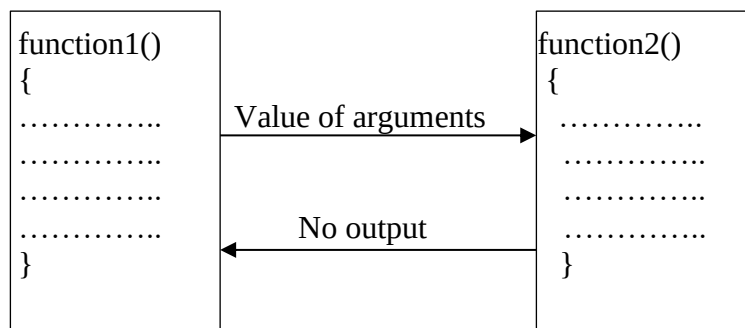
int a,b ,sum;
printf("Enter 2 numbers ");
scanf(" %d %d",&a ,&b);
sum=a+b;
printf(" Sum is %d",sum);
}

```

Here **addition** is the user defined function which is the called function, main is the calling function which calls the user defined function. Since there are no arguments the calling function does not pass any input to the called function, and the return type is void so the called function does not return the output to the calling function. So there is no communication between the called function and the calling function.

Category 2 : Functions with arguments & no return values

When a function has arguments then it receives the data from the calling function. We can make the calling function to read data from the terminal and pass it to the called function. When it does not return a value the calling function does not receive any data from the called function. The communication between the called and the calling function with arguments and no return value is shown in the figure.



Example :

```

void addition (int a, int b)
main ()
{
int a, b;
printf("enter 2 numbers")
scanf("%d%d", &a,&b);
addition(a,b);
return 0;
}
void addition(int a,int b)
{
int sum;
sum=a+b;

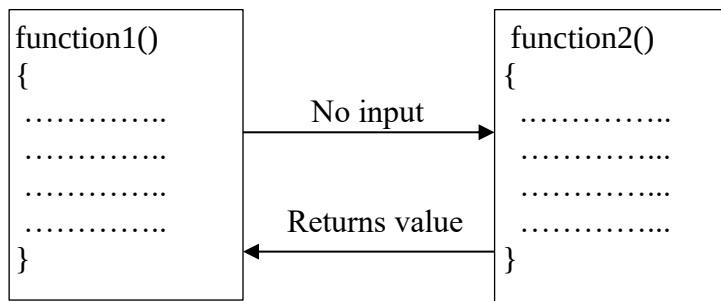
printf("Sum is %d",sum);
}

```

Here **addition** is the user defined function which is the called function, main is the calling function which calls the user defined function. Two arguments(input values) are read from the terminal in the calling function and passed to the called function, and the return type is void so the called function does not return the output to the calling function.

Category 3 : Functions with no arguments but returns a value

When a function has no arguments it does not receive any data from the calling function. When there is return type that means that the called function returns the output to the calling function which will then print the output. The communication between the called and the calling function with arguments and no return value is shown in the figure.

**Example :**

```

int addition():
main()
{
int sum;
sum=addition();
printf("Sum is %d",sum);
return 0;
}
int addition()
{
int a,b,sum;
printf("Enter 2 numbers");

```

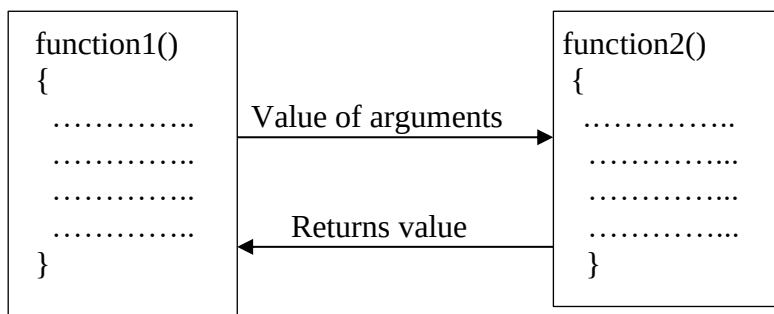


```
scanf("%d%d",&a,&b);
sum=a+b;
return sum;
}
```

Here **addition** is the user defined function which is the called function, main is the calling function which calls the user defined function. Since there are no arguments the calling function does not pass any input to the called function. But there is a return value that means the called function returns the output to the calling function which will be then stored in the variable sum and the value of sum is printed.

Category 4 : Functions with arguments & return values

When a function has arguments then it receives the data from the calling function. We can make the calling function to read data from the terminal and pass it to the called function. When there is return type that means that the called function returns the output to the calling function which will then print the output. The communication between the called and the calling function with arguments and no return value is shown in the figure.



Example :

```
int addition(int a, int b)
main()
{
int a,b,sum;
printf("Enter two numbers");
scanf("%d%d",&a,&b);
sum=addition(a,b);
printf ("sum is %d",sum);
return 0;
}
int addition(int a, int b)
{
int sum;
sum=a+b;
return sum;
}
```

Here **addition** is the user defined function which is the called function, main is the calling function which calls the user defined function. Two arguments(input values) are read from the terminal in the calling function and passed to the called function. Since there is a return value that means the called function returns the output to the calling function which will be then stored in the variable sum and the value of sum is printed.

6) Write a note on a) nesting of functions b) recursive functions.

Nesting of functions

A function that is called inside another function is called as the nested function. A Function can be called inside the body of another function. main can call function1, which calls function2, which calls function3 and so on. When a called function calls another function a process of “chaining” occurs. There is no limit on how deeply functions can be nested.

Example:

```
void India( );
void America( );
main( )
{
printf(“Hello world\n”);
india( );
return 0;
}
void India( )
{
printf(“I love my India\n”);
America( );
}
void America( )
{
printf(“America America”);
}
```

Output will be like this:

```
Hello world
I love my India
America America
```

Here we have declared 2 functions India() and America(). main is the executable function where the function India() has been called. Inside the function India() the second function America() has been called. Both these functions are the nested functions since they have been called inside another function and both will be executed.

Recursion

The process of function calling itself is called as recursion. The function that calls itself is called as the recursive function.

Recursive functions can be used to solve problems where solutions are expressed by applying the same solutions repeatedly. When we write recursive function then we need to have an if statement to force the function to return. Otherwise the function will not return at all, it will repeatedly get executed.

Let us take a simple example of a recursive function :

```
main()
{
printf(“ This is an example for recursion\n”);
main();
}
```

When this is executed, the output will be something like this:

```
This is an example for recursion
```

This is an example for recursion

This is an example for recursion

.

.

Here the execution need to be terminated abruptly, otherwise the execution will continue indefinitely.

Another example for recursion is the factorial program where an if statement is used to terminate the recursive function by returning the value when the solution is met.

Program to calculate factorial of 'n' integers using recursion.(Important)

```
#include<stdio.h>
int factorial(int);
main()
{
int n,fact;
clrscr();
printf("Enter an integer:");
scanf("%d",&n);
fact=factorial(n);
printf("Factorial of %d is %d",n,fact);
getch();
return 0;
}
int factorial(int n)
{
if(n==0 || n==1)
return 1;
else
return(n*factorial(n-1));
}
```

7) Explain with example defining a structure, declaring structure variable and accessing structure members.

Structure is a constructed datatype that is used to group the elements of different datatypes.

Declaring structure variable

A structure variable declaration is similar to the declaration of variables of any other datatype.

It includes the following elements:

1. The keyword struct
2. The structure tag name
3. List of variable names separated by commas
4. A terminating semicolon

General form :

struct tag_name variable1,variable2;

Example:

struct book_bank book1,book2;

It declares book1 and book2 as variables of type **struct book_bank**.

Each one of these variables has three members as specified by the template. The complete declaration will look like this:

```

struct book_bank
{
    char title[20];
    int pages;
    float price;
};
struct book_bank book1,book2;

```

Here book1 and book2 are structure variables of struct book_bank datatype. title, pages and price are the members of the variables book1 and book2. Members of the structure are not variables. They do not occupy any memory until they are associated with the structure variables such as **book1**.

It is also allowed to combine both the structure definition and variables declaration in one statement.

The declaration

```

struct book_bank
{
    char title[20];
    int pages;
    float price;
} book1,book2;

```

The use of tag name is optional here.

Here it declares book1 and book2 as structure variables representing 2 books but does not include tag name. Structure definition appears at the beginning of the program before any function or variables are declared.

Defining a structure

Structures must be defined first that can be used later to declare structure variables.

General form of structure definition is :

```

struct tag_name
{
    data_type    member1;
    data_type    member2;
    .....
    .....
};

```

- The template is terminated with a semicolon.
- Entire definition is considered as a statement, each member is declared independently for its name and type in a separate statement inside the template.
- The tag name can be used to declare structure variable of its type.

Example :

```

struct book_bank
{
    char title[20];
    int pages;
    float price;
};

```

The keyword struct declares a structure to hold the details of three fields, namely title,pages and price. These fields are called as structure members. Each member may belong to different data type. book_bank is the name of the structure which is called as the structure tag. The tag name is used to declare variables.

The above definition has not declared any variables. It simply describes format called template to represent information.

Accessing structure members

The members themselves are not variables. They should be linked to the structure variables in order to make them meaningful members. For example word title has no meaning whereas “title of book2” has meaning. The link between a member and a variable is established using the member operator ‘.’ which is also known as dot operator. For example

book1.price

is the variable representing the price of book1 and can be used as any other ordinary variable.

Here how we can assign values to the members of book1:

book1.title = “ Programming in C”;

book1.price=120.50;

will assign the title “Programming in C” and price 120.50 to book1.

We can also use scanf to give values through the keyword. For example

scanf(“%s”, book1.title);

scanf(“%f”, book1.price);

These are valid input statements.

8) Explain with the help of code example array of structures.

An array of structures in C can be defined as the collection of multiple structures variables where each variable contains information about different entities. The array of structures in C are used to store information about multiple entities of different data types. The array of structures is also known as the collection of structures.

Let's see an example of an array of structures that stores information of 5 students and prints it.

```
#include<stdio.h>
#include <string.h>
struct student{
    int rollno;
    char name[10];
};
main(){
    int i;
    struct student st[5];
    printf("Enter Records of 5 students");
    for(i=0;i<5;i++){
        printf("\nEnter Rollno:");
        scanf("%d",&st[i].rollno);
        printf("\nEnter Name:");
        scanf("%s",&st[i].name);
    }
    printf("\nStudent Information List:");
    for(i=0;i<5;i++){
        printf("\nRollno:%d, Name:%s",st[i].rollno,st[i].name);
    }
    return 0;
}
```

Here struct is the keyword, student is the structure tag or the name of the structure. Roll no and name are the members of the structures. struct student will be the user defined data type that declares st as the array of 5 elements. st[0], st[1], st[2], st[3] and st[4] are the 5 elements of array st. Structure

members are accessed by linking the structure member and the variable name using the dot(.) operator. For example st[0].rollno is used to access the rollno of the first student, st[1].name is used to access the name of the second student and so on. Records of 5 students are read continuously using scanf() with the help of for loop. Same records are printed or displayed. An array of structures is stored inside the memory in the same way as a multi dimensional array.

st[0].rollno	101
st[0].name	Venita
st[1].rollno	102
st[1].name	Renita
st[2].rollno	103
st[2].name	Frenita
st[3].rollno	104
st[3].name	Jenita
st[4].rollno	105
st[4].name	Benita

9) Explain with code example array within structure.

A structure may contain elements of different data types – int, char, float, double, etc. It may also contain an array as its member. Such an array is called an array within a structure. An array within a structure is a member of the structure and can be accessed just as we access other elements of the structure.

Arrays may be the member within structure, this is known as arrays within structure. Accessing arrays within structure is similar to accessing other members.

```
#include<stdio.h>
#include<string.h>
struct Student
{
int Roll;
char Name[10];
int marks[3]; //array of marks
};
main()
{
int i;
struct Student S;
printf("Enter Student Roll : ");
scanf("%d",&S.Roll);
printf("Enter Student Name : ");
scanf("%s",&S.Name);
printf("Enter the marks in 3 subjects");
for(i=0;i<3;i++)
scanf("%d",&S.marks[i]);
}
printf("Roll : %d",S.Roll);
printf("Name : %s",S.Name);
for(i=0;i<3;i++)
printf("Marks in 3 subjects: %d",s.marks[i]);
}
```

This program uses the concept of the array within a structure. The program displays the record of a student comprising the roll number, name, and marks secured in 3 subjects. The marks in various subjects have been stored under an array called marks. Rolln, name and marks are the

members of the structure student. S is the variable name with user defined data type struct Student. Structure members are accessed by linking the structure member and the variable name using the dot(.) operator. S.Rollno is used to access the rollno of the student, S.name is used to access the name of the student and S.marks[i] is used to access the marks of 3 subjects since marks is an array member. Rollno, name and marks in 3 subjects are read using scanf() statement. for loop is used to read the marks in 3 subjects since it is an array member. These records are then displayed or printed.

10) Write a note on nesting of structures.

Nested structure in C is nothing but structure within structure. One structure can be declared inside other structure as we declare structure members inside a structure.

For example, we may need to store the address of an entity employee in a structure. The attribute address may also have the subparts as street number, city, state, and pin code. Hence, to store the address of the employee, we need to store the address of the employee into a separate structure and nest the structure address into the structure employee. Consider the following program.

```
#include<stdio.h>
struct address
{
    char city[20];
    int pin;
    char phone[14];
};
struct employee
{
    struct address add;
};
main ()
{
    struct employee emp;
    printf("Enter employee information?\n");
    scanf("%s %s %d %s",emp.name,emp.add.city, &emp.add.pin, emp.add.phone);
    printf("Printing the employee information....\n");
    printf("name: %s, City: %s, Pincode: %d, Phone: %s",emp.name,emp.add.city,emp.add.pin,emp.add.phone);
}
```

In the above example, structure address is defined first with members city, pin & phone. This is a normal structure i.e. there is no nesting. But, the structure student having member add is **nested structure** because the member add is type of another user created structure. So, structure student is nested because of member add.

- This program explains how to use structure within structure in C using normal variable. “address” structure is declared inside “employee” structure in this program. Both structure variables are normal structure variables.
- Please note that members of “employee” structure are accessed by 2 dot(.) operator and members of “address” structure are accessed by single dot(.) operator.

11) Explain with example defining a union, declaring union variable and accessing union members.

Unions

Unions follow the same syntax as structures. Instead of the keyword struct, keyword union is used.

General form (Syntax):

```
union tag_name
{
    data_type    member1;
    data_type    member2;
    .....
    .....
};
```

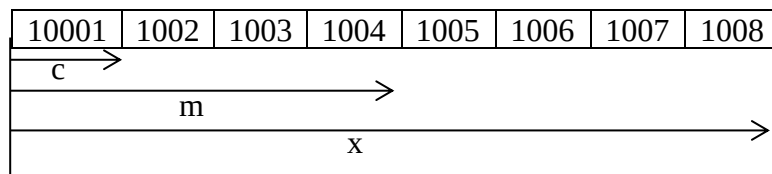
There is only one difference in terms of storage. In structures each member has its own storage location, whereas all the members of union share the same storage location. That implies that although a union contains many members with different types, it can handle only one member at a time.

Union is declared using the keyword union.

Example :

```
union item
{
    int m;
    double x;
    char c;
}
union item store;
```

This declares a variable store of datatype **union item**. The union contains three members m, x and c each with different types. But unlike structures we can use only one member at a time since only one location is allocated for a union variable irrespective of its size.



Compiler allocates storage that is large enough to hold the largest variable type. Here the largest variable type is double that needs 8 bytes. Then the other variables share the same location.

To access union member, we can use the same syntax as the structures.

```
code.m
code.x
```

Example :

```
code.m=109;
code.x=9.9822;
printf("%d",code.m);
printf("%lf",code.m);
```

Storage can be used by any one of the members at a time. When a different member is assigned a new value it will take the place of the previous member's value.

Initialization can be done as follows:

```
union item store = {100};
```

this is valid because the first member is of int type.

```
union item store = {'V'};
```

is invalid. Always the value of the first type needs to be initialized first.